

Architecture



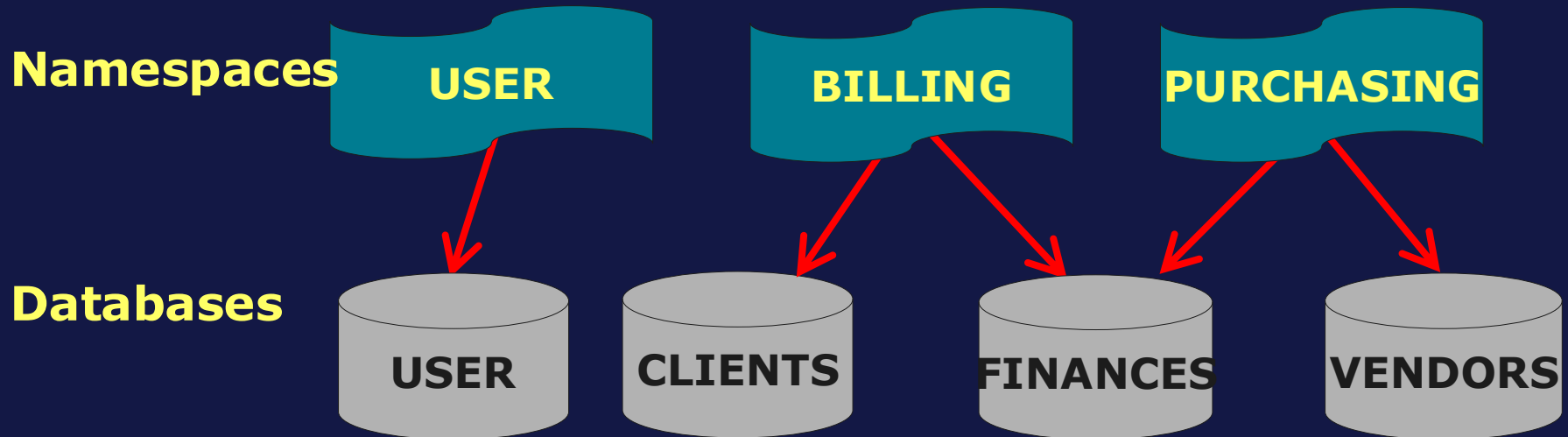
Overview

- Namespaces and databases.
- Globals and routines.
- System databases.
- Naming conventions.



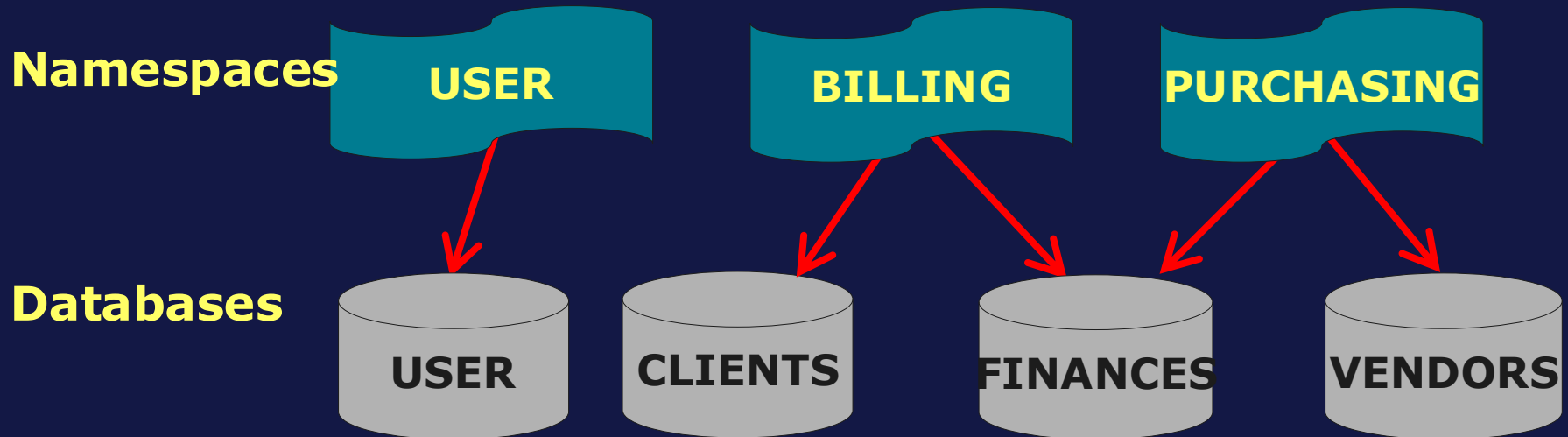
Basic Structure

- Data and code are stored in databases, referred to by namespaces.
 - Database is physical storage location.
 - Namespace defines logical references to several databases.

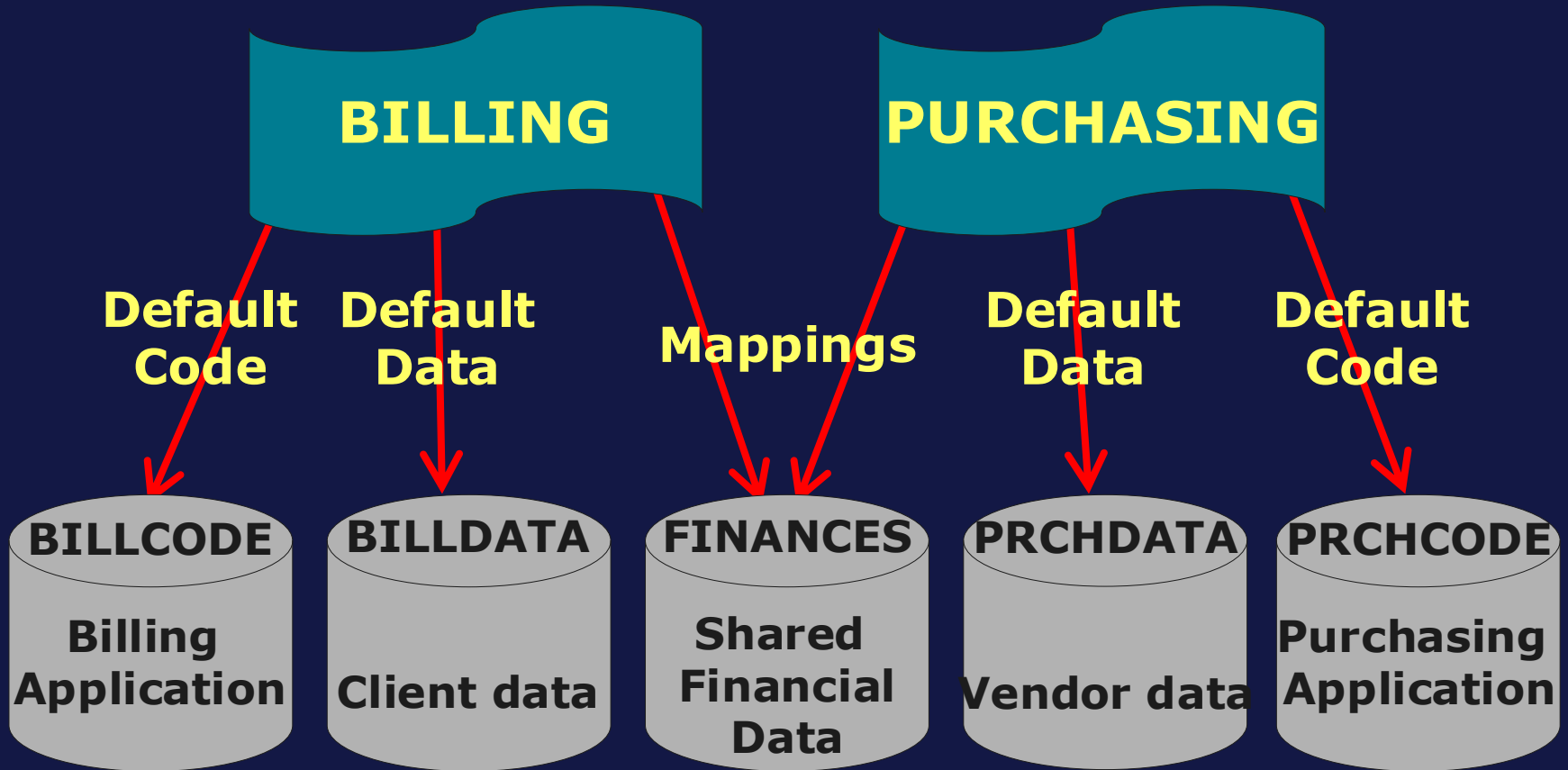


Basic Structure (cont.)

- Several namespaces can refer to same database.
- Each namespace must have at least one default (primary) database, for data and code.

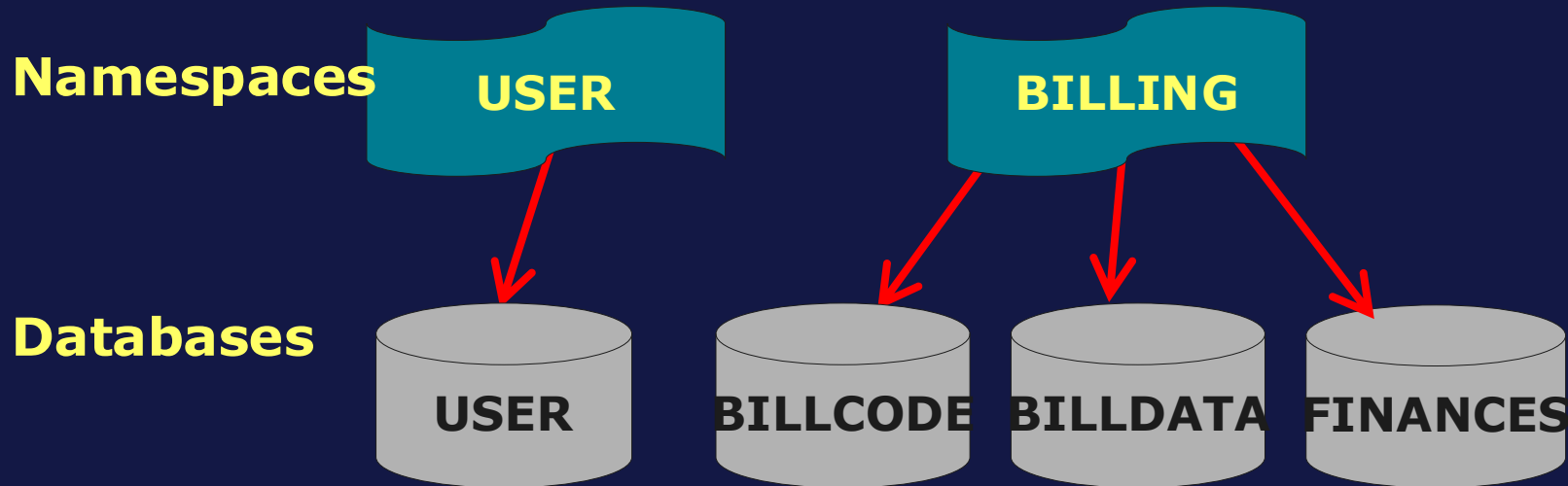


Example



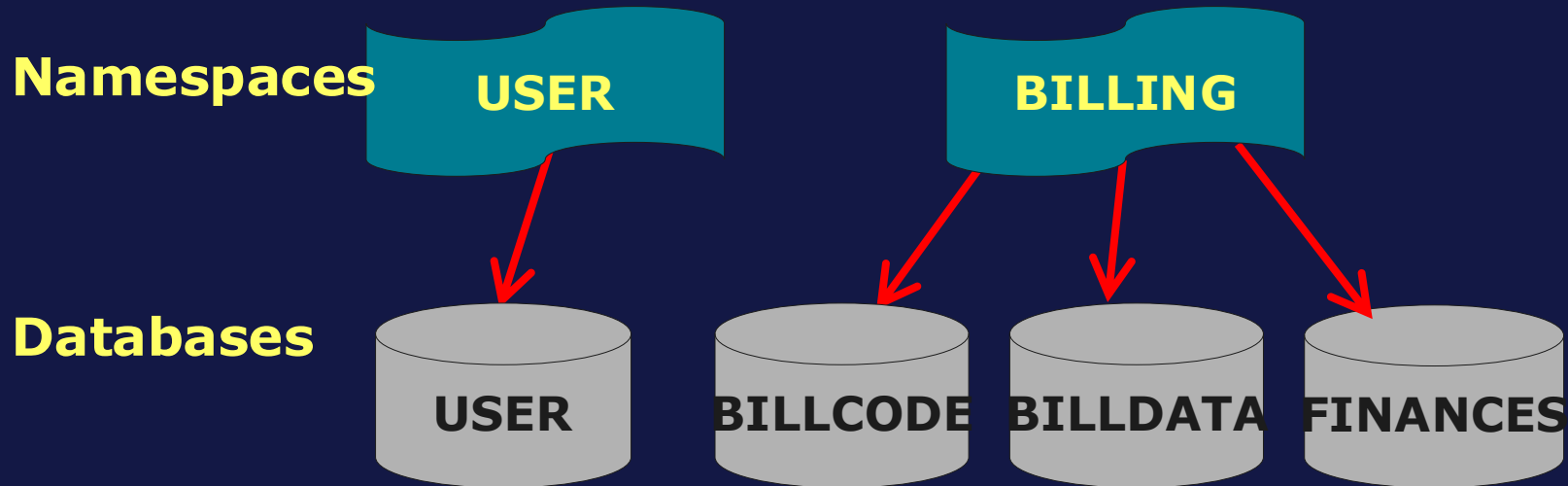
Namespaces

- Each application process runs in one namespace.
- Process accesses data and code in databases referenced by that namespace.



Namespace Benefits

- Allows code and data to be distributed among multiple databases and servers.
- Allows code in one database to access code and data in other databases, regardless of physical location.



Databases

- Each database represents IRIS.DAT file in OS file system.
- Database locations:
 - System databases located within installation directory.
 - Place application databases outside of installation directory.
- Maximum size of single database file:
 - Default: 32TB, for 8 KB database block size
 - **ExtendedDatabaseSize:** 8PB, for 8 KB block
- Database contains code and/or data.



How To: Pick Namespace

- Terminal:

- `set $namespace = "Namespace"`
- `zn "Namespace"`
- `do ^%CD` and enter namespace at prompt.

- VS Code:

- Create separate Workspace for each namespace.

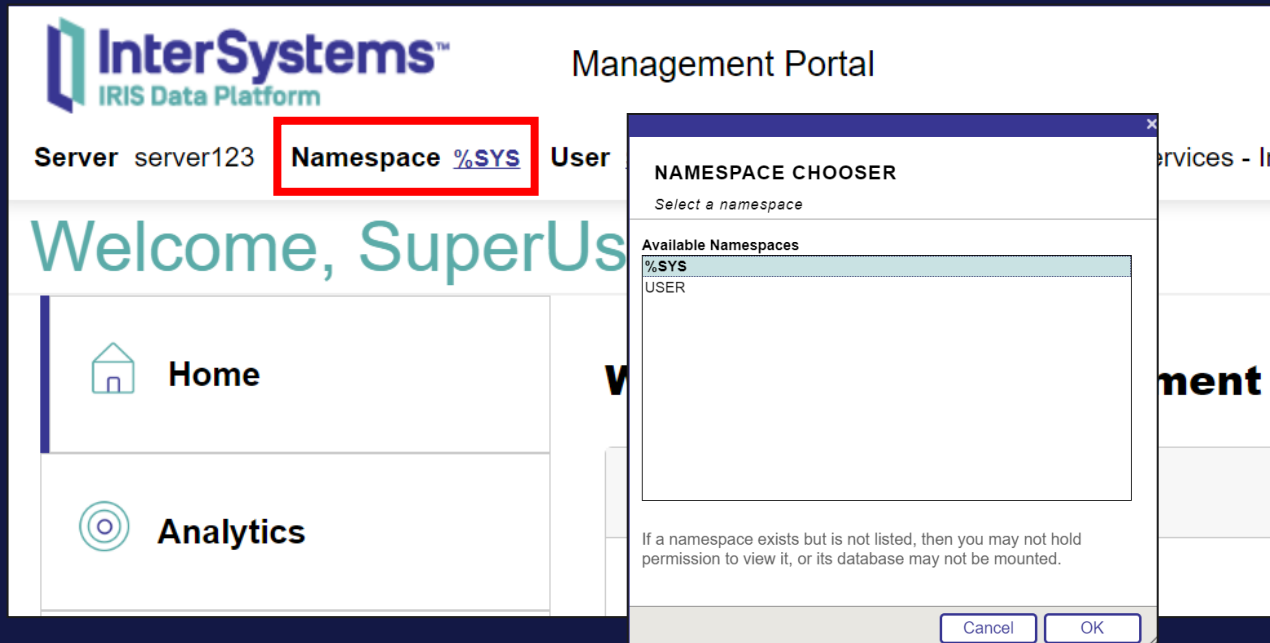
- Studio:

- File > Change Namespace.



How To: Pick Namespace (cont.)

- Management Portal:
 - Click name of current namespace near top of Home page.
 - For some Management Portal pages choose namespace from drop-down list on left.
 - Pre-v2024.1, click Switch near top of Home page.



Default Databases for Namespace

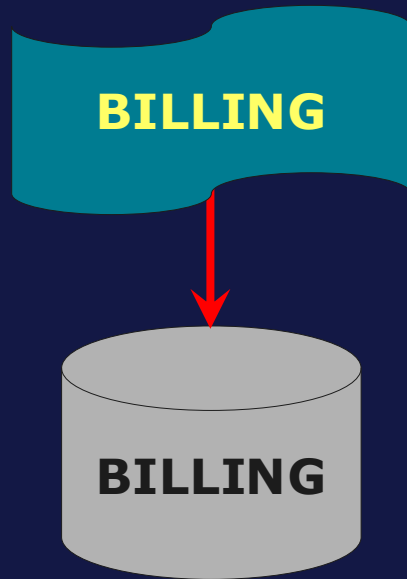
Edit Properties for Namespace BILLING

Name	BILLING
	Required.
Default Database for Globals	BILLDATA
Default Database for Routines	BILLCODE
Default Database for Temporary Storage	IRISTEMP

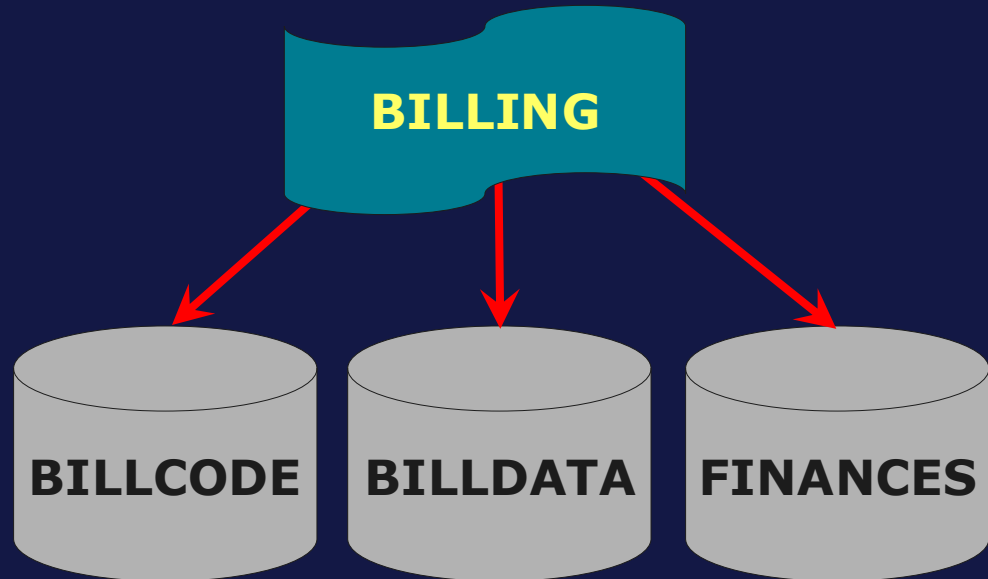
- Database for Globals:
 - Stores data.
- Database for Routines:
 - Stores code, including code in classes.



Mapping Data and Code



BILLING: Contains all data and application code.



- **BILLDATA:** Client data.
- **BILLCODE:** Billing Application code.
- **FINANCES:** Financial data. (Global Mapping)



All Data Stored In Globals

- Persistent multidimensional arrays.
 - Global names start with ^.
 - ^abc is a global.
 - ^abc (1, 2, 3) is particular location in global (global reference or global node).
 - Can store all data types including character and binary streams.
- Globals starting with ^% are available system-wide.
 - Accessible in any namespace.
 - Mapped from IRISSYS and IRISLIB.



Flexibility of Globals

- Enables support of multiple models.
 - Object, relational (including columnar storage), document, vector, multidimensional, XML.
- System also stores code for routines and classes in globals.



View Global Data

- Management Portal:
 - System Explorer > Globals > select global > View.
- Terminal:
 - `do ^%G`
 - `zwrite ^global`



All Code Runs In Routines

- Routine code is written by hand or generated from class definitions.
- Routines starting with % are available system-wide.
 - Runnable in any namespace.
 - Mapped from IRISSYS.
 - Typically for InterSystems routines.
- Classes in packages starting with % are available system-wide.
 - Mapped from IRISSYS and IRISLIB.



Class/Routine Extensions

Extension	Description
.cls	Class definition.
.inc	File containing macro definitions used in ObjectScript code.
.int, .mac	Routines containing ObjectScript code.
.obj	Executable code.



Managing Globals and Routines

- Data and code management primarily affects globals and routines.
 - Managing caches for globals and routines:
 - System Configuration module.
 - Creating global, routine and package mappings for data and code access:
 - Application Configuration module.
 - Journaling changes to globals:
 - Journaling module.
 - Compiling statistics for globals and routines:
 - System Monitoring module.



View Code

- View routine code in Management Portal:
 - System Explorer > Routines > select routine > Code.
- View (and edit) classes and routines using IDEs.



Installation Databases

IRIS
IRISSYS
IRISLIB*
IRISAUDIT
IRISTEMP*
IRISLOCALDATA*
IRISMETRICS (v2024.3+)
IRISSECURITY (v2025.2+)
USER
ENSLIB*

- IRIS for Health contains these additional databases.

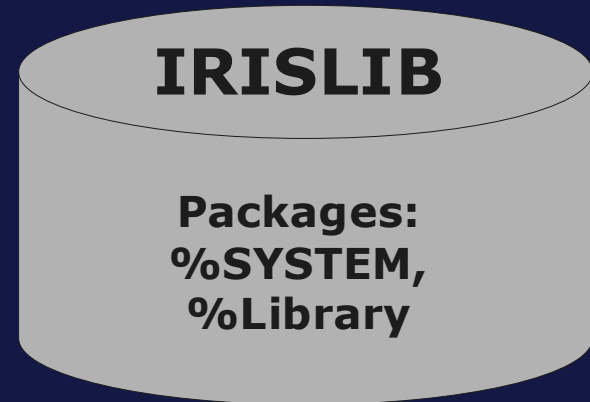
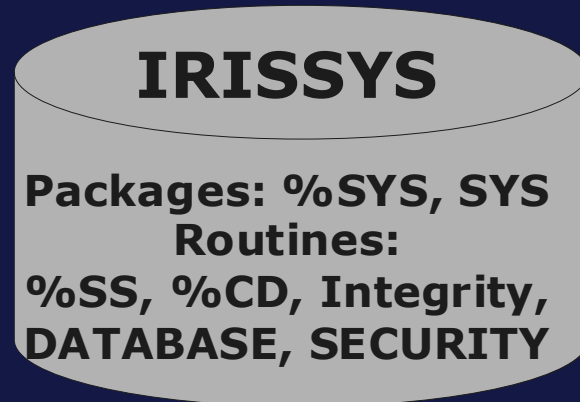
IRIS for Health
HSSYS
HSLIB*
HSCUSTOM
HSSYSLOCALTEMP*

*** Database completely replaced during upgrade.**



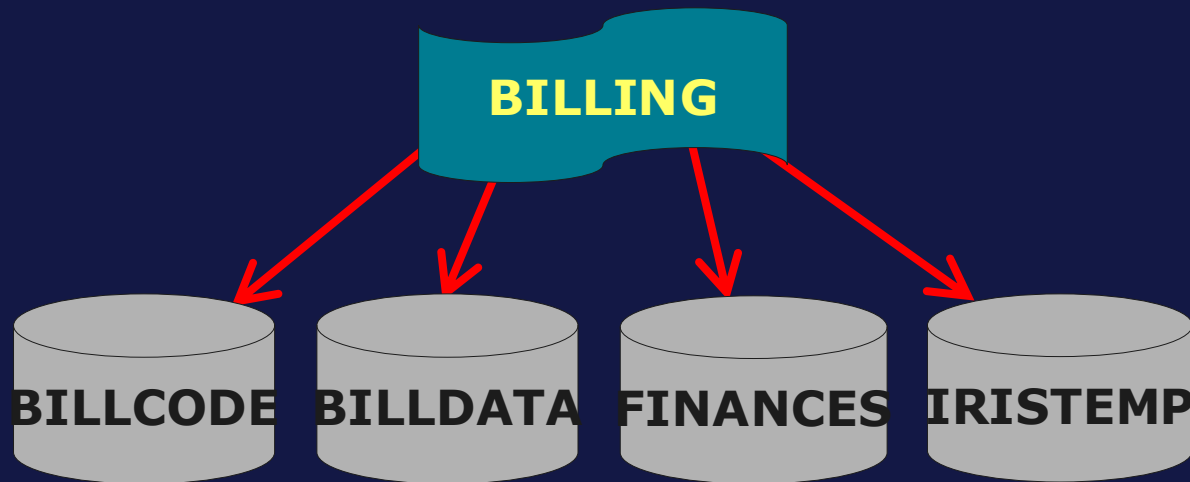
System and Library Databases

- IRISYS contains low-level system management code and data.
 - Access highly restricted.
- IRISLIB contains system-wide code and data.
 - Read-only access.
- Examples of database content:



Temporary Data

- IRISTEMP holds temporary or “scratch” globals.
 - For example, report with multiple aggregations keeps them in temporary global until report completes.
- Tries to keep data in database cache on server, avoiding writing to disk.

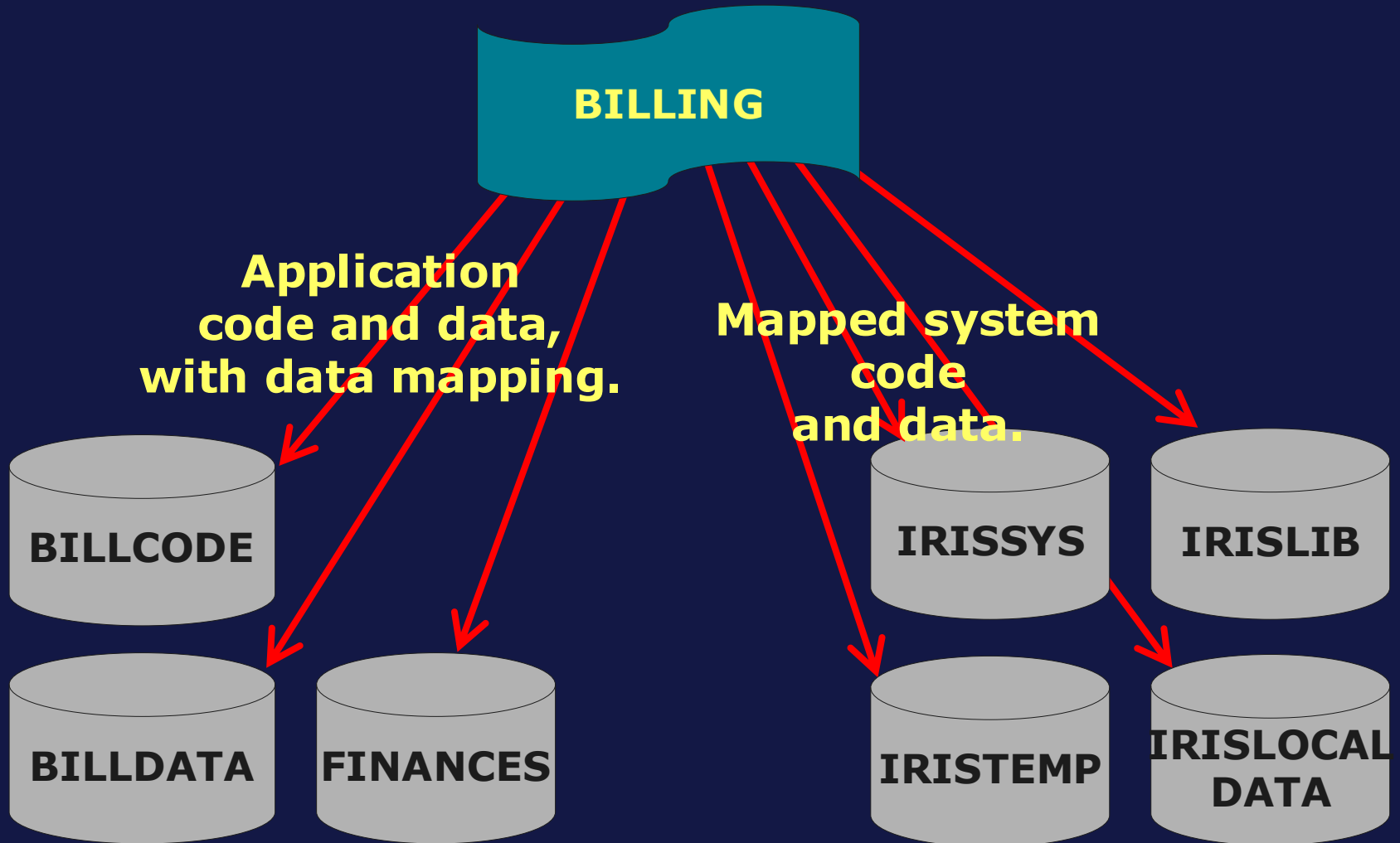


Namespace Default Mappings

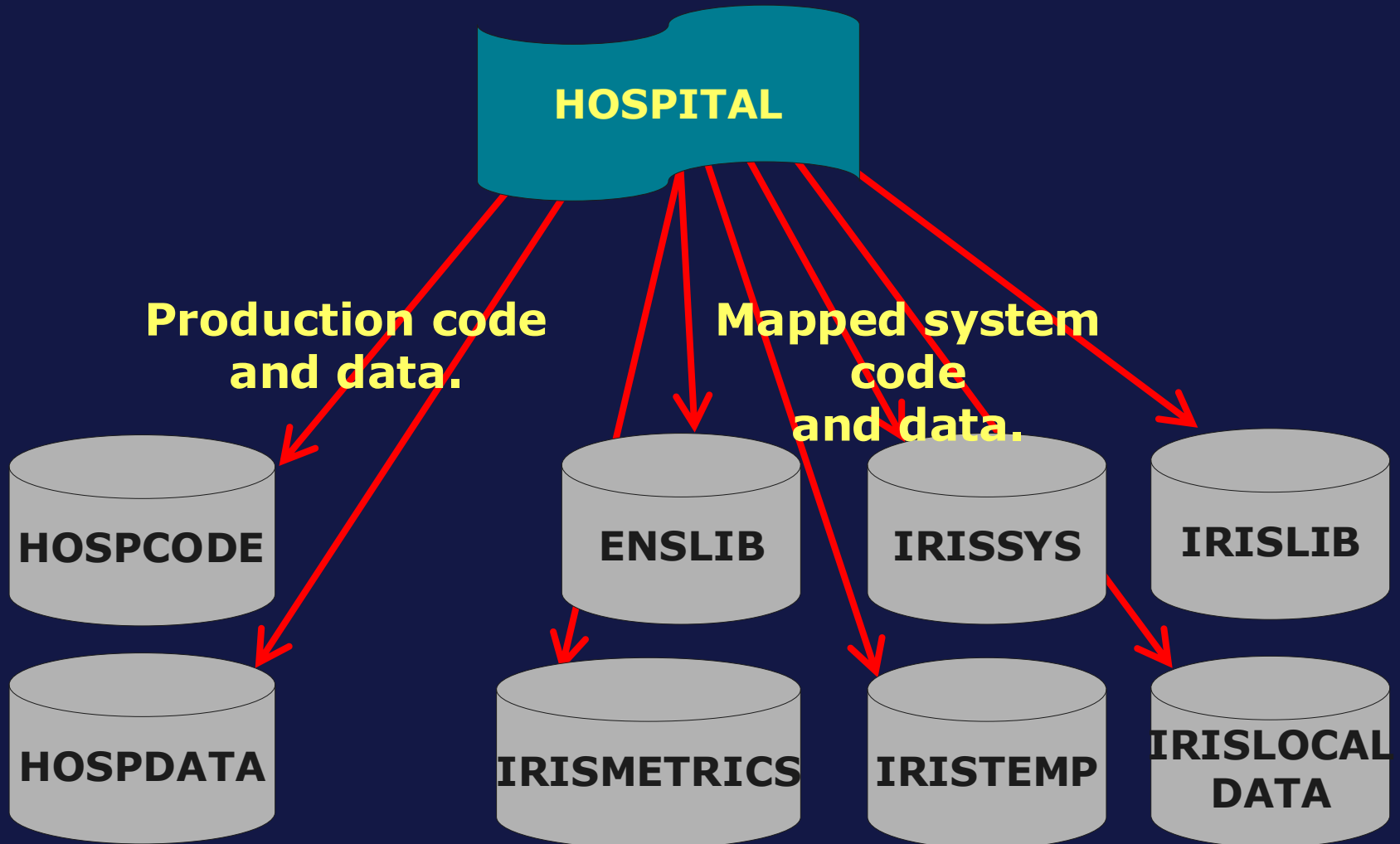
InterSystems IRIS System Databases	
IRISSYS	
IRISLIB	
IRISTEMP	
IRISLOCALDATA	
Interoperability-enabled Namespaces Only	
ENSLIB	
IRISMETRICS	



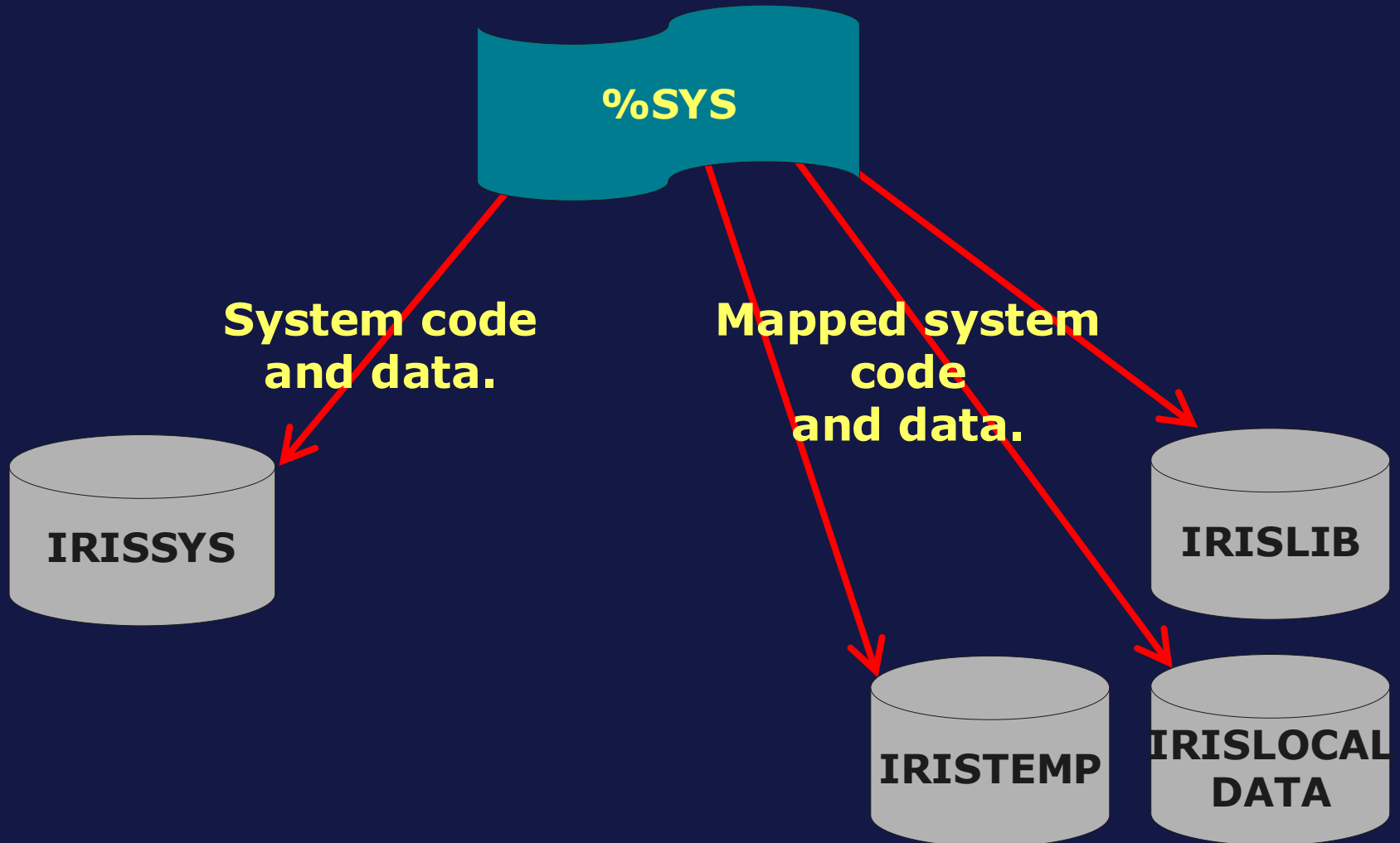
Example: Namespace Mappings



Example: Interoperability Production Namespace Mappings



%SYS Namespace



Naming Conventions

- Database and Namespace names.
 - Capital letters or numbers.
 - First character must be letter or %.
 - Database name cannot start with %.
 - May contain _ and - characters.
- Global, routine, package and class names.
 - Case-sensitive letters or numbers.
 - First character must be letter or %.
 - Class name cannot start with %.



Hands-on

- On **irisa** server
 - Create a new namespace "Training"
 - With 2 databases
 - TrainingCode database for code
 - TrainingData database for data



Suggested Readings

- *System Administration Guide*
 - *Configuring InterSystems IRIS.*
 - *Configuring Namespaces.*
 - *Configuring Databases.*



Configuring the System



Overview

- Database and routine caches.
- Memory allocation.
- Journal settings.
- Configuration files.



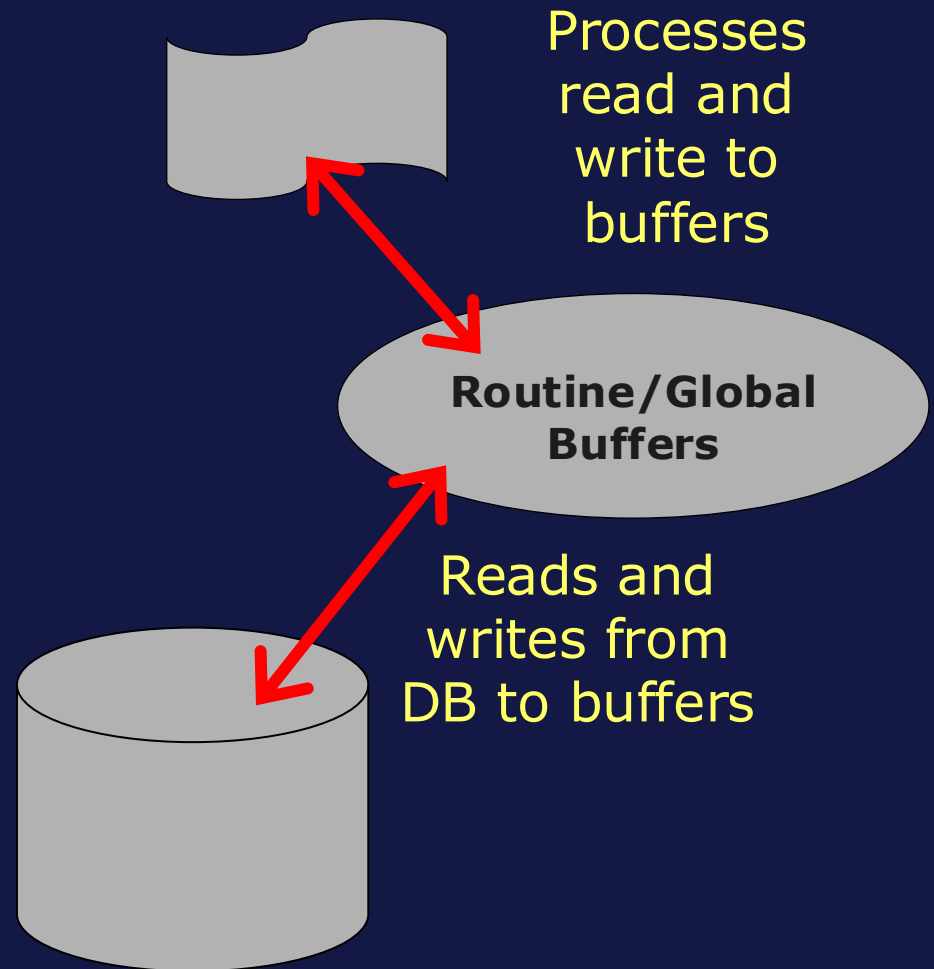
Database Block Size

- By default, IRIS database files use 8 KB blocks.
- Larger block sizes are available when creating new databases.
 - 16 KB, 32 KB, and 64 KB.
 - Contact InterSystems Worldwide Response Center to consider larger block sizes for:
 - Big-string data and/or large binary strings.
 - ECP configurations with data elements larger than 4 KB.
 - Disk systems that work more efficiently with larger block sizes.



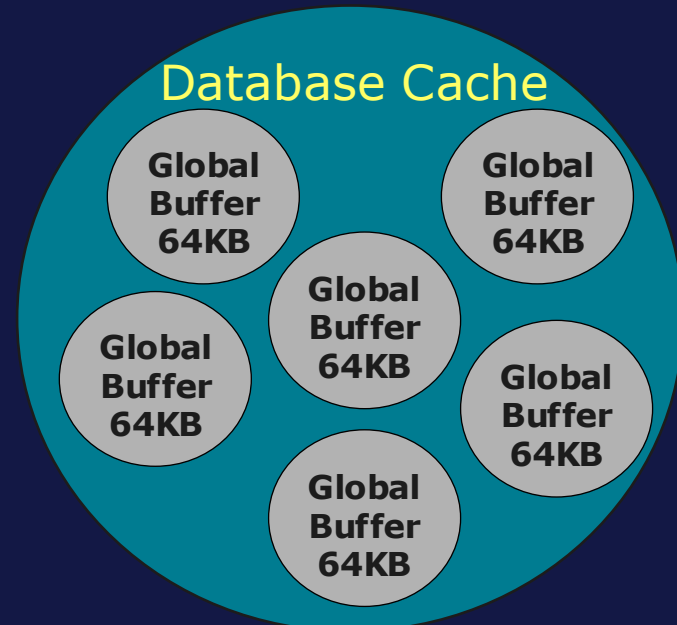
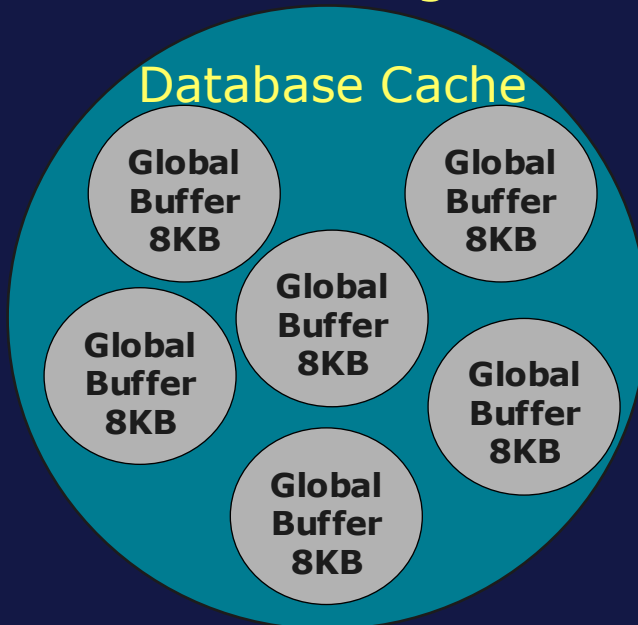
Buffers

- Buffer is section of memory.
 - Stores globals and routines for access by running processes.
 - Accessing information from buffer instead of disk can be 1000-1000000 times faster.



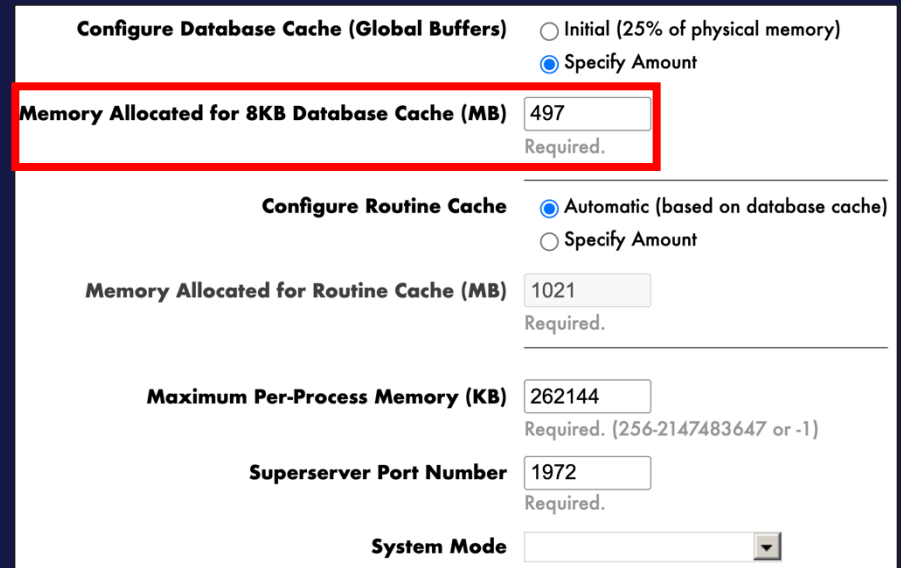
Database Cache

- Global buffer sizes must support configured database block sizes.
- Separate database cache for each block size.
- Always loads entire data block.
 - Uses smallest global buffer that fits database block.



Database Cache (cont.)

- Adjust total size of database cache.
 - System Administration > Configuration > System Configuration > Memory and Startup.
 - Default not appropriate for most environments.
- Allocate for each type of database block used.
 - Bigger is better up to a point.



Configure Database Cache (Global Buffers) ☐ Initial (25% of physical memory) ☒ Specify Amount

Memory Allocated for 8KB Database Cache (MB) Required.

Configure Routine Cache ☒ Automatic (based on database cache) ☐ Specify Amount

Memory Allocated for Routine Cache (MB) Required.

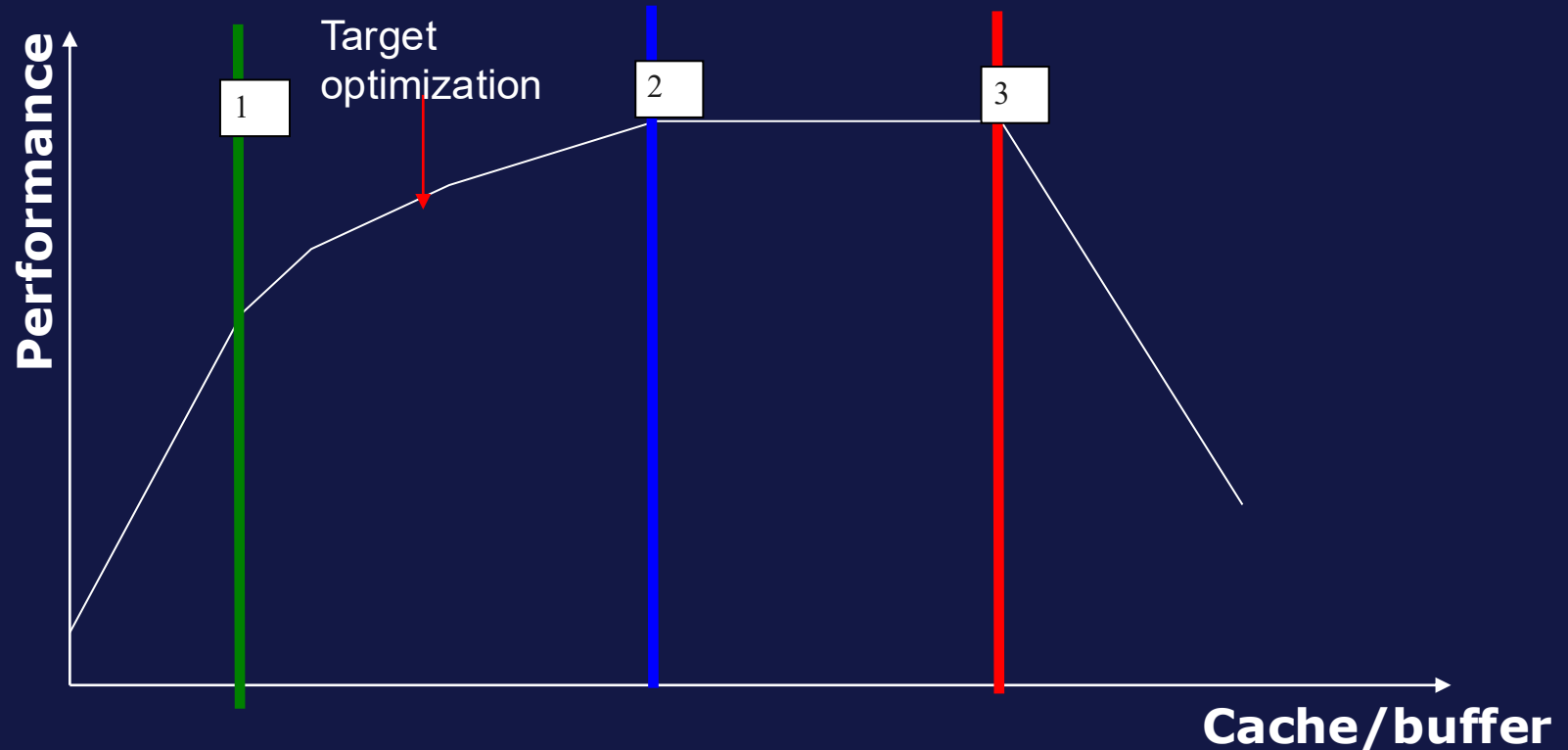
Maximum Per-Process Memory (KB) Required. (256-2147483647 or -1)

Superserver Port Number Required.

System Mode



Tuning Database Cache



Tuning Database Cache

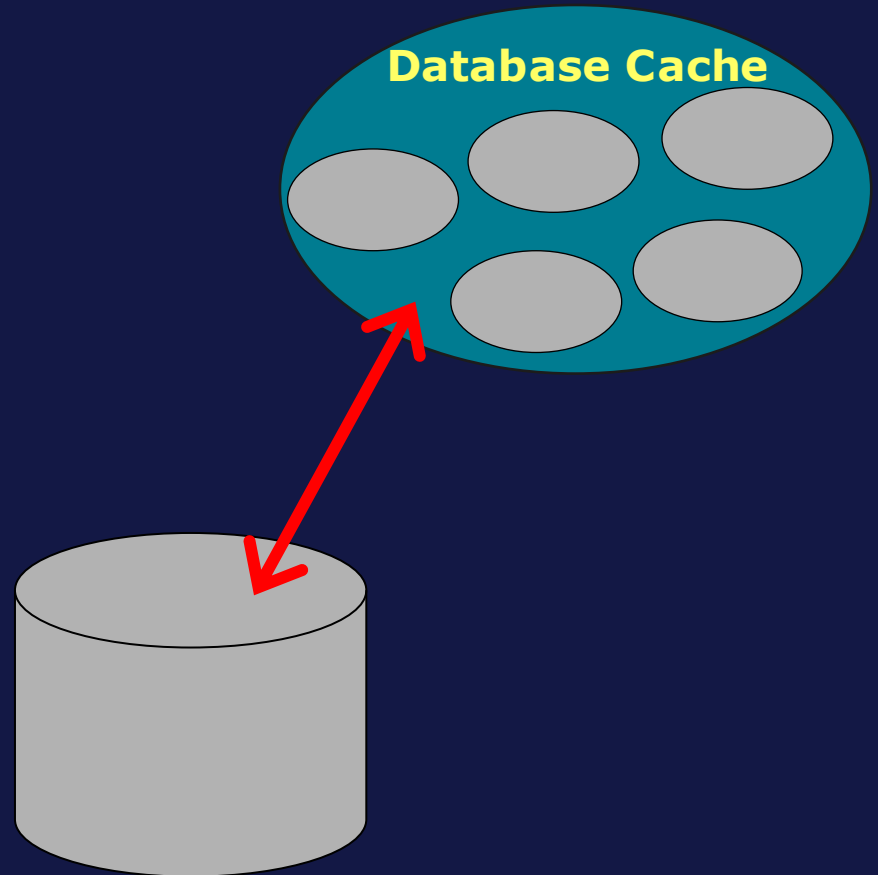
- While running applications, record Cache Efficiency.
 - $\frac{\text{Global References}}{\text{Disk Reads} + \text{Writes}}$ since system start.
- View using:
 - System Operation > System Dashboard > System Performance.
 - %SYS>do ^GLOSTAT.
 - Can also collect average value over specified timeframe.

SYSTEM PERFORMANCE	
Globals/Second:	41.00
Global Refs:	15,430,341,007
Global Sets:	2,648,864,217
Routine Refs:	4,506,896,721
Logical Requests:	6,357,588,871
Disk Reads:	1,405,849
Disk Writes:	6,324,176
Cache Efficiency:	1,996.16



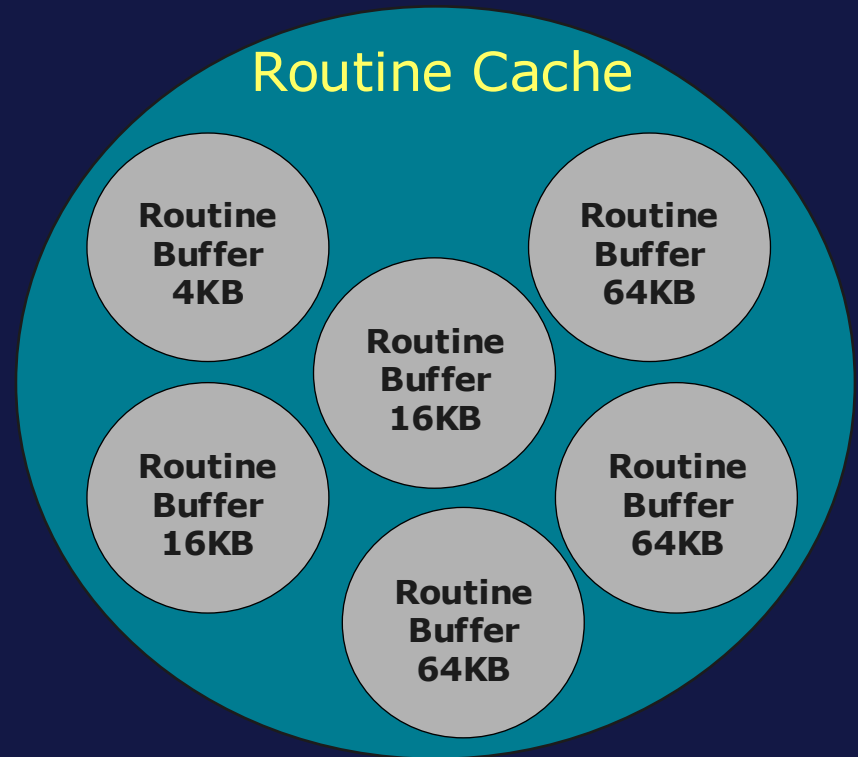
Tuning Database Cache (cont.)

- Higher Cache Efficiency is better.
 - Value appropriate if applications running well without complaints.
 - If not, increase size of database cache.
 - Benchmark value over time.



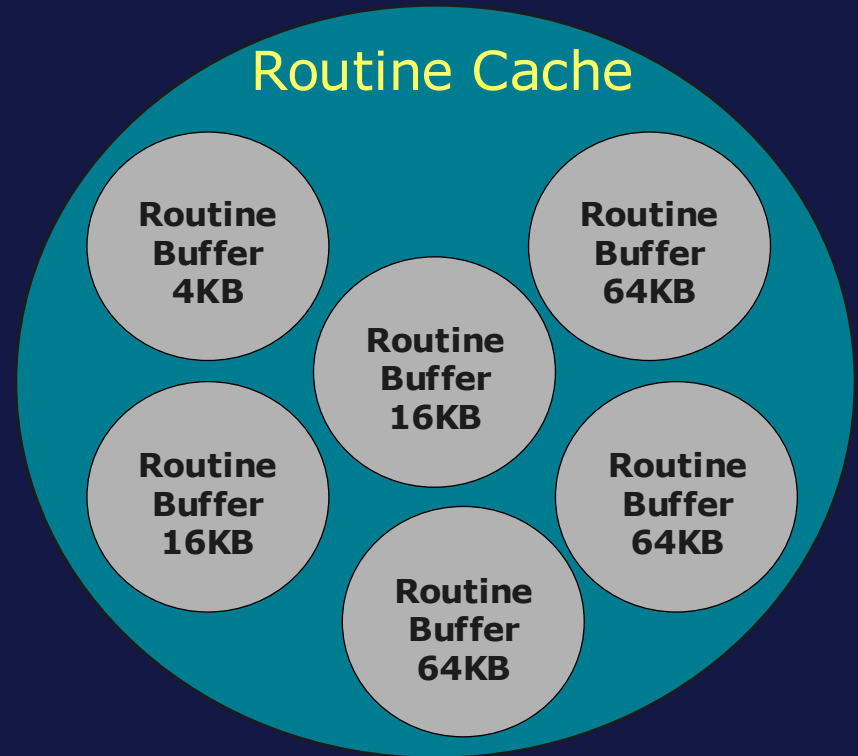
Routine Cache

- Routine cache contains routine buffers.
 - Processes share one copy of each routine.
- System Administration > Configuration > System Configuration > Memory and Startup.



Routine Cache Buffer Sizes

- Defaults to use 4KB, 16KB and 64KB buffers.
 - 8KB and 32KB also possible to configure.
- Uses smallest available buffer that fits routine.
 - For larger routines (up to 8MB), multiple 64KB buffers used.



Tuning Routine Cache

- Goal is cache large enough to hold maximum # of simultaneous routines running.
- While running applications, record Routine buffer loads and saves:
 - Count of loads/saves of routines since system start.
- View using:
 - System Operation > System Usage.
 - %SYS>do ^GLOSTAT.
 - Can also collect counts per second over specified timeframe.
- Routine buffer loads and saves < 20/second for 60 seconds is okay.
 - 0 is ideal.



Setting Cache Sizes

- Automatically (default).
- Manually.
 - Changing values requires restart.
- Carefully evaluate cache size allocation.

Configure Database Cache (Global Buffers)	☒ Initial (25% of physical memory) ☐ Specify Amount
Memory Allocated for 8KB Database Cache (MB)	497 Required.
Configure Routine Cache	☒ Automatic (based on database cache) ☐ Specify Amount
Memory Allocated for Routine Cache (MB)	80 Required.



Automatic Allocation of Cache Sizes

Cache	Allocation	Minimum	Maximum
Database	25% of total physical memory	32MB	16TB
Routine	Approximately 10% of amount allocated to database cache	80MB	1020MB



Buffer Allocations Within Routine Cache

- Default allocation of routine cache:
 - 1/2 for 64KB buffers, 3/8 for 16KB buffers and 1/8 for 4KB buffers.
 - For example, 500MB routine cache would have:
 - 3906 64KB buffers (250MB).
 - 11718 16KB buffers (187.5MB).
 - 15625 4KB buffers (62.5MB).
 - Minimum of 430 buffers allocated for each size (35MB total).
- Default allocation usually appropriate.
 - Check with InterSystems Worldwide Response Center if allocations need tuning.



Process Memory

- Each process uses memory for:
 - Stacks – string, frame and argument.
 - Current routine and local variables.
 - Other process information.
- Memory for each process starts at 128KB.
 - Each time process needs more memory, system doubles allocated memory.
 - If doubling memory allocation surpasses the maximum allowed process memory, system generates <STORE> error.



Maximum Per-Process Memory

- Absolute maximum is 2TB.
- Default maximum limit: -1, which means 2TB.
 - Could allow multiple large processes to exhaust server memory.
- Change maximum using System Administration > Configuration > System Configuration > Memory and Startup > Maximum Per-Process Memory.



Shared Memory Heap

- Part of memory used by processes and internal memory structures including:
 - Process table.
 - Lock table.
 - Audit system.
 - Mirroring.
- Default value.
 - 0. Automatically allocates 3% of amount allocated to global cache (v2023.1+).
- Change value using System Administration > Configuration > Additional Settings > Advanced Memory > gmheap.
 - Requires restart.



Shared Memory Heap (cont.)

- If need to change setting, methods provided to analyze value:
 - `write $SYSTEM.Config.SharedMemoryHeap.GetUsageSummary()`
 - `write $SYSTEM.Config.SharedMemoryHeap.FreeCount()`
 - `write $SYSTEM.Config.SharedMemoryHeap.MaxCount()`
- **System Operation > System Usage > Shared Memory Heap Usage** has current values.



Lock Table Size

- Memory allocated for managing locks for concurrency.
- System Administration > Configuration > Additional Settings > Advanced Memory > locksiz.
- Default value:
 - 0. Allows size to dynamically increase/decrease within shared memory heap. (v2023.1+)
 - 16 MB. (pre-v2023.1)



Lock Table Full

- If Lock Table fills, message written to messages log.
 - As processes release locks, space becomes available again.
- To prevent reoccurrence:
 - If using dynamic allocation, increase Shared Memory Heap.
 - By 16-32MB, or more if necessary.
 - If **not** using dynamic allocation:
 - Double or triple current Lock Table size.
 - Increase Shared Memory Heap by same amount.



Total Memory Requirements

- Keep total memory requirements within physical memory of machine.
 - Database and Routine Caches.
 - Shared memory heap.
 - Processes.
 - Operating system.
 - Non-InterSystems applications.
- Exceeding physical memory causes excessive paging:
 - Can degrade performance.



Memory Allocation

- Shared Memory Heap.
 - In v2023.1+, start with default allocation (3% of global cache).
 - Otherwise, start with 64MB multiplied by number of cores.
- Routine Cache.
 - Ideally allocate enough to hold all code needed for application.
 - 1020MB usually sufficient.
 - Tune using Routine buffer loads and saves metric.



Memory Allocation (cont.)

- Database Cache.
 - ~50%-80% of memory.
 - Higher value for application that performs many data reads.
 - Lower value for application that performs mostly writes with fewer reads.
 - Tune using Cache Efficiency metric.
- Remaining memory available for:
 - System and user processes.
 - OS (~2GB memory).
- Application may require adjustments to memory allocations.
 - Benchmark performance over time.



Sample Memory Allocations

	Database Cache	Routine Cache	Shared Memory Heap	Available for OS and Processes
128GB 12 cores	102GB	1020MB	3GB (v2023.1+) or 768MB	22-24GB
64GB 8 cores	44GB	1020MB	1.3GB (v2023.1+) or 512MB	18-19GB
32GB 4 cores	16GB	1020MB	512MB (v2023.1+) or 256MB	15GB



OS and Memory Considerations

- When possible lock caches in memory.
 - Details are OS-specific.
 - Configure Large pages in Windows and IBM AIX®.
 - Configure Huge pages in Linux.
- Other OS-level memory settings also affect performance.
- Check *Installation Guide* for OS-specific recommendations.



Configuration Parameter File (CPF)

- CPF file contains configuration settings defined in Management Portal.
 - InstallDirectory/iris.cpf.
 - **Does not** contain any security settings.
- To modify CPF file:
 - Management Portal.
 - Text editor.
- Invalid CPF file prevents system from starting.
 - To validate CPF file:

```
%SYS>do  
##class(Config.CPF).Validate("PathAndNameOfFile",...)
```



Activating CPF File

- Many configuration changes can be activated while system running.
 - To manually activate while running:

```
%SYS>do ##class(Config.CPF).Activate()
```



Multiple CPF Files

- Can create multiple CPF files.
- Specify which file to use when starting.
`iris start InstanceName CPFName`
- Automatically created:
 - `_LastGood_.cpf` contains last known good CPF file used by system.
 - `*.cpf_yyyymmdd` contains copy of first CPF file saved for day.
- To view name of CPF file being used:
 - Home > About > Configuration field.
 - `iris list` (conf file field).



CPF Merge Feature

- Modify existing CPF using configuration merge file.
 - Partial CPF that sets desired values for defined parameters.
- Can use at startup or to reconfigure running instance.
 - At startup (non-Windows only):
 - `ISC_CPF_MERGE_FILE=/merge_files/merge.cpf iris start InstanceName`
 - On running instance:
 - `iris merge InstanceName [merge-file] [target-cpf]`



CPF File Security

- Determines if changing CPF file **from OS** requires username/password.
 - Prompted for username/password if:
 - Start system with CPF file different from last one used.
 - New CPF file activated on running system.
 - Username must have %Admin_Manage:Use.
 - If no username/password provided, uses previous or existing CPF file.
- System Administration > Security > System Security > System-wide Security Parameters > Enable configuration security.



Changing Parameters

- Many parameters should not be changed under normal operating conditions.
- Verify impact of change beforehand.
 - *Additional Configuration Settings Reference* documents all parameters.
 - Help link for any setting brings up definition in documentation.

GlobalVectors

Number of global vectors allocated for each process. The default is 32 global vectors. The range is 2–32 global vectors. InterSystems recommends you do not adjust this parameter. See [MaxMemPerProcess](#).

IJCBufferSize

Number of bytes allocated for each InterJob Communication (IJC) buffer. The default is 512 bytes. The range is 0–8192 bytes. For details, see the “[Interprocess Communication](#)” chapter in the *Caché I/O Device Guide*. Also see [IJCDevices](#).

If you edit this setting, you must restart Caché to apply the change.

LineRecallBuffer

Total size (in bytes) of all input strings to be stored in the command



CPF File Documentation

- InterSystems IRIS.
 - *Configuration Parameter File Reference.*

Chapter 1:	Introduction to the InterSystems Parameter File
1.1	Filename and Location
1.2	InterSystems IRIS Configurations
1.3	Specifying a CPF File
1.4	Editing the Active CPF File
1.5	Configuration Security
1.6	Parameter File Format
1.6.1	Spaces
1.6.2	Sections
1.6.3	Parameters
1.7	Parameter Descriptions
1.8	List of Sections and Parameters



Suggested Readings

- Developer Community Article:
 - *InterSystems Data Platforms and Performance Part 4 - Looking at Memory*
- *System Administration Guide*
 - *Configuring InterSystems IRIS > Configuring Data.*
- *System Resource Planning and Management.*
- *Configuration Parameter File Reference.*
- *Security Administration Guide.*



Summary

- What are the key points for this module?

